

# Mixtures of Deterministic Programs in Probabilistic Coherence Spaces

Anonymous Author(s)

Anonymous Institution

**Abstract.** Probabilistic coherence spaces are the standard denotational model of higher-order probabilistic programs. We ask which of their elements are mixtures of deterministic ones. For each type we take the mixtures of its certain states, the elements with values in  $\{0, 1\}$ , as its probability calculus. At every formula of multiplicative-additive linear logic over finite data types, the certain states are cliques of Girard's coherence space and the calculus lies inside the probabilistic coherence space. The two agree at first order and differ from second order on. For a program with two functional arguments, the calculus is the stable set polytope of a graph whose clique-constrained polytope is the coherence space, and five web points separate them, with the bounds of the KCBS inequality. For the dual of a tensor of two additive types, the two coincide iff one of the two coherence graphs has no induced four-cycle, and the gap transfers to every formula in which such a tensor occurs negatively. Linear negation exchanges the calculus with a local calculus cut out by the certain states of the dual, and the coherence space is the self-dual set between them. The calculus is not compositional: first-order types are tight, but their tensor is not. At the dual of the second-order type, the gap is Bell's theorem, witnessed by the Popescu-Rohrlich box. All results are mechanized in Lean 4.

**Keywords:** Probabilistic coherence spaces · Linear logic · Probabilistic programs · Perfect graphs · Contextuality · Lean

## 1 Introduction

*Mixtures of determinism.*

*Coherence spaces model.*

*Definability question.*

*Certain-state calculus.*

*First order agrees.*

*Second-order gap.*

*Exact condition.*

*Transfer.*

*Duality, Bell.*

*Quantum contrast.*

*Mechanization.* Every result is mechanized in Lean 4 over `mathlib` [5,1], except steps the text marks as not formalized. A superscript <sup>Lean</sup> links a statement to its Lean counterpart in Appendix A, which also describes the development.

*Contributions.*

1. **Cliques.** At every multiplicative–additive formula, the certain elements of the coherence space are cliques of the coherence space and incoherent pairs carry total weight at most 1, so Girard’s intuition holds throughout. The converse fails at  $(T \otimes T)^\perp$  through Shannon’s set.
2. **Second order.** At  $T$  the calculus is STAB and the coherence space is QSTAB of one exclusivity graph. An explicit pentagon point lies outside the calculus, and with three arguments there is a gap among always-halting programs.
3. **Exact condition.** For formulas built from data types with  $\&$  and  $\oplus$ , the calculus and the coherence space of  $(X \otimes Y)^\perp$  coincide iff one coherence graph has no induced 4-cycle. The mechanized proof avoids perfect graph theory and extends Ravindra’s theorem.
4. **Transfer.** Coordinate retracts carry the gap to every formula with a negative occurrence of a tensor whose factors both have induced 4-cycles. Beyond second order, a gap also exists without such a tensor.
5. **Duality and Bell.** Linear negation exchanges the calculus and the local calculus, and the two are tight iff neither the formula nor its dual has a gap. Tightness holds at every  $\mathbf{n} \multimap \mathbf{m}$  and fails at the tensor of two of them, so the certain-state calculus is not compositional. At the core the gap is Bell’s theorem, equivalent to the pentagon gap at  $T$ .
6. **Mechanization.** All of the above is in Lean 4 over `mathlib`, with no `sorry` and standard axioms. The results of this paper take about 9,200 lines in 29 files. The general theorems are proved for arbitrary finite data types, and the finite facts behind the witnesses (the five-cycle of the pentagon point, the support of the Popescu–Rohrlich box, and the bases of the Kochen–Specker set) are checked by evaluation on explicit finite types (Appendix A).

## 2 Calculi from Certain States

We describe a construction scheme that takes as input a set of events together with a set of assignments that settle every event, and returns the set of mixtures of these assignments. The construction and its completeness theorem are de

Finetti's [3], in the form that Paris and Williams use for non-classical logics [6,7]. All functions take values in the nonnegative reals  $\mathbb{R}_{\geq 0}$ .<sup>1</sup>

## 2.1 Presentations and Mixtures

**Definition 1 (presentation, calculus<sup>Lean</sup>).** *A presentation is a set  $E$ , whose elements are the events, together with a set  $S$  of functions  $E \rightarrow \mathbb{R}_{\geq 0}$  with values in  $\{0, 1\}$ , the certain states. A mixture of functions  $s_1, \dots, s_n$  is a function  $\sum_i w_i s_i$  with weights  $w_i \in \mathbb{R}_{\geq 0}$  and  $\sum_i w_i = 1$ . The calculus  $\text{Mix}(S)$  is the set of mixtures of certain states. A set of functions is closed under mixtures if it contains every mixture of its elements.*

Mixtures are finite. When  $E$  is finite, so is  $S$ , and a countable mixture of elements of  $\text{Mix}(S)$  regroups into a finite mixture of certain states, so countable mixtures would change nothing. When  $E$  is infinite they differ. If  $S$  contains the indicators of the singletons  $\{e_1\}, \{e_2\}, \dots$ , then  $\sum_n 2^{-n} 1_{\{e_n\}}$  takes infinitely many values, while a finite mixture of  $\{0, 1\}$ -valued functions takes finitely many, so it is not in  $\text{Mix}(S)$ .

A certain state settles every event, with value 1 or 0. A mixture records uncertainty about which certain state obtains, with  $w_i$  the weight of  $s_i$ . In Section 3 the events are the points of the web of a formula of linear logic and the certain states are the deterministic behaviours of that type, so an element of the calculus is a probability distribution over deterministic behaviours.

**Theorem 1 (certain elements and mixing<sup>LeanLean</sup>).** *For every presentation  $(E, S)$ : (i) a function with values in  $\{0, 1\}$  lies in  $\text{Mix}(S)$  iff it lies in  $S$ , and (ii)  $\text{Mix}(\text{Mix}(S)) = \text{Mix}(S)$ , so  $\text{Mix}(S)$  is the least set that contains  $S$  and is closed under mixtures.*

For (i), let  $x = \sum_i w_i s_i$  have values in  $\{0, 1\}$ , and let  $w_j \neq 0$ . At an event  $e$  with  $x_e = 0$ , every  $s_i$  with  $w_i \neq 0$  has  $s_i(e) = 0$ . At an event with  $x_e = 1$ , every such  $s_i$  has  $s_i(e) = 1$ , since  $\sum_i w_i = 1$  and  $s_i(e) \leq 1$ . So  $s_j = x$ . Part (ii) expands a mixture of mixtures into one mixture with product weights. Mixing therefore creates no new certain states, and the certain states of the calculus are those of the presentation.

## 2.2 Affine Laws and Completeness

The calculus has an axiomatic description by linear inequalities.

<sup>1</sup> The Lean development works in the extended nonnegative reals  $[0, \infty]$  of `mathlib`, the type in which measures take values. Every function in this paper takes finite values, so the statements are the same, except that some Lean statements carry a finiteness hypothesis that is automatic over  $\mathbb{R}_{\geq 0}$ , as in Theorem 4.

**Definition 2 (affine law<sup>Lean</sup>).** Let  $E$  be finite. An affine law on  $E$  is given by functions  $a, b : E \rightarrow \mathbb{R}_{\geq 0}$  and constants  $k, k' \in \mathbb{R}_{\geq 0}$ . A function  $x : E \rightarrow \mathbb{R}_{\geq 0}$  satisfies it if

$$\sum_{e \in E} a_e x_e + k \leq \sum_{e \in E} b_e x_e + k'.$$

An affine law is valid on a set of functions if every function in the set satisfies it.

Every linear inequality with real coefficients between the values of  $x$  can be written in this form, by moving negative coefficients to the other side. For example,  $x_e \leq 1$ ,  $x_e + x_{e'} \leq 1$ , and the equation  $x_e + x_{e'} = x_{e''}$ , as two inequalities, are affine laws.

**Theorem 2 (completeness [3,6]<sup>LeanLean</sup>).** Let  $E$  be finite and  $S$  a set of functions  $E \rightarrow \mathbb{R}_{\geq 0}$  with values in  $\{0, 1\}$ . Then  $\text{Mix}(S)$  is the set of functions  $x : E \rightarrow \mathbb{R}_{\geq 0}$  that satisfy every affine law valid on  $S$ .

Affine laws are preserved by mixtures, which gives the inclusion from left to right. For the converse, a point outside the convex hull of the finite set  $S$  is separated from it by a hyperplane, and the hyperplane is an affine law valid on  $S$  that the point violates. The Lean proof uses the geometric Hahn–Banach theorem of `mathlib`. For rational points and certain states, the development also contains a proof without the axiom of choice by Fourier–Motzkin elimination, the elimination of variables from a system of linear inequalities<sup>Lean</sup>.

Read as bets, an affine law valid on  $S$  and violated at  $x$  is a finite system of bets, priced by  $x$ , that loses money in every certain state. Paris and Williams use Theorem 2 in this form for non-classical logics, and Williams derives from it that the mixtures are the assignments that no other assignment dominates in accuracy [6,7].

### 2.3 Laws of Deterministic Programs

The calculus is determined by its certain states and their laws.

**Theorem 3 (uniqueness<sup>LeanLean</sup>).** Let  $E$  be finite, let  $S$  be a set of functions  $E \rightarrow \mathbb{R}_{\geq 0}$  with values in  $\{0, 1\}$ , and let  $C$  be a set of functions  $E \rightarrow \mathbb{R}_{\geq 0}$  that contains  $S$ . (i)  $C$  satisfies no affine law that some element of  $S$  violates. If  $C$  satisfies every affine law valid on  $S$ , then the affine laws valid on  $C$  are exactly those valid on  $S$ . (ii) If  $C$  is closed under mixtures and satisfies every affine law valid on  $S$ , then  $C = \text{Mix}(S)$ .

Part (i) holds because  $S \subseteq C$ . For (ii), closure under mixtures gives  $\text{Mix}(S) \subseteq C$ , and Theorem 2 gives  $C \subseteq \text{Mix}(S)$ . So  $\text{Mix}(S)$  is at once the least set that contains the certain states and is closed under mixtures and the largest set that obeys their laws. The mathematics is de Finetti’s and Paris’s [3,6].

In the rest of the paper  $C$  is the probabilistic coherence space of a formula, which contains its certain states and is closed under mixtures, and  $\text{Mix}(S)$  is the calculus of the same formula. The question is which of the two cases of the following definition holds.

**Definition 3 (gap).** *Let  $(E, S)$  be a presentation and  $C \supseteq S$  a set of functions  $E \rightarrow \mathbb{R}_{\geq 0}$  that is closed under mixtures. Then  $\text{Mix}(S) \subseteq C$ , and  $C$  has a gap over  $S$  if the inclusion is strict.*

**Corollary 1 (a gap is a violated law<sup>Lean</sup>).** *For finite  $E$ , the set  $C$  of Definition 3 has a gap over  $S$  iff some element of  $C$  violates an affine law valid on  $S$ .*

This is Theorem 2 applied to the elements of  $C$  outside  $\text{Mix}(S)$ . When the certain states are the deterministic programs of a type, an affine law valid on  $S$  is an inequality that every deterministic program obeys, and every probabilistic program that flips coins and then runs a deterministic program obeys it too. A gap is an element of the probabilistic coherence space that violates such an inequality. The witnesses found below are of this kind: a five-term inequality at the type of a program with two functional arguments, with the bounds of the KCBS inequality (Section 4), and the CHSH inequality at the dual type (Section 7).

## 2.4 Why Not Lattice Valuations

For intuitionistic and classical logic, the calculus of a finite event algebra is also given by the familiar axioms of a valuation: a grading  $v$  of the events with  $v(\perp) = 0$  and  $v(\top) = 1$  that is monotone and *modular*,  $v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b)$ , where  $\sqcup$  and  $\sqcap$  are the join and meet of the algebra of events [6]<sup>Lean</sup>. The event algebras of linear logic are *quantales*, complete lattices with an associative multiplication  $\otimes$  that distributes over arbitrary joins and need not be idempotent [4]. The tensor  $\otimes$  is not the meet, so modularity does not apply to it as it stands. Replacing the meet by the tensor in the modularity axiom does not work either.

**Definition 4 (tensor valuation<sup>Lean</sup>).** *Let  $\Omega$  be a join-semilattice, a partial order in which every two elements have a least upper bound  $a \sqcup b$ , and let  $\otimes$  be a binary operation on  $\Omega$ . A tensor valuation for  $\otimes$  is a function  $v : \Omega \rightarrow \mathbb{R}_{\geq 0}$  such that  $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$  for all  $a, b \in \Omega$ .*

**Theorem 4 (tensor collapse<sup>LeanLean</sup>).** *Let  $\Omega$  be a join-semilattice,  $\otimes$  a binary operation on  $\Omega$ , and  $v$  a tensor valuation for  $\otimes$ . Then  $v(a \otimes a) = v(a)$  for every  $a \in \Omega$ .*

Setting  $b = a$  in the defining equation and using  $a \sqcup a = a$  gives  $v(a) + v(a) = v(a) + v(a \otimes a)$ , and  $v(a)$  cancels. No property of  $\otimes$  is used.

*Example 1 (a constant valuation<sup>Lean</sup>).* On  $\mathbb{N}$  with  $\sqcup = \max$  and  $\otimes = +$ , an associative operation that distributes over  $\max$  and has  $1 + 1 \neq 1$ , setting  $a = 0$  in the defining equation gives  $v(n) = v(0)$  for every  $n$ , so every tensor valuation is constant.

So a grading of the events of a quantale by this law cannot separate one copy of a resource,  $a$ , from two copies,  $a \otimes a$ , which is the distinction that the tensor of linear logic exists to make. Probabilistic coherence spaces make the distinction by assigning weights to the points of a web, with  $a \otimes a$  interpreted on a product of webs, and not by grading formulas [2]. The construction of this section applies to them, because it needs a set of events and a set of certain states and no operations on events.

### 3 Linear Logic Background

*(from sec-recipe §10.3 and §10.5)*

#### 3.1 Coherence spaces, data types, first order

#### 3.2 MALL formulas and their semantics

#### 3.3 Coherence at every formula

#### 3.4 The exponential !Bool (credit Danos–Ehrhard)

### 4 Second Order

*(from sec-second-order §11.1)*

#### 4.1 Sequential programs and Kuhn

#### 4.2 A language for $T$

#### 4.3 The pentagon gap

#### 4.4 Divergence and total programs (Baumeler–Wolf, $T_3$ )

### 5 The Exact Condition

*(from §11.4)*

#### 5.1 STAB and QSTAB

#### 5.2 Clique property and perfection

#### 5.3 Four-cycles: a direct proof

#### 5.4 Beyond second order

### 6 Other Formulas

*(from §11.2)*

## 6.1 Coordinate retracts

## 6.2 Negative occurrences and four-cycles

# 7 Local Certainty and Duality

(from §12.2, §11.3, §13.2)

## 7.1 The local calculus

## 7.2 A clique outside the coherence space (Shannon)

## 7.3 Negation, tightness, and the tensor (Cor cor:cat-tensor)

## 7.4 Bell’s theorem at the core

# 8 Quantum Event Logic

(from §12)

## 8.1 Kochen–Specker for Cabello’s set

## 8.2 Divergence and the stable-set bound

## 8.3 Local certainty and Born states

# 9 Related Work

*Probabilistic coherence spaces and full abstraction.* Ehrhard–Pagani–Tasson, Castellan et al.

*Generalized probabilism.* Paris, Williams, Gyenis.

*Contextuality.* Cabello–Severini–Winter, Acín–Fritz–Leverrier–Sainz, Abramsky–Brandenburger.

*Causal processes.* Baumeler–Wolf, Kissinger–Uijlen, Simmons–Kissinger.

*Antiblocking duality and perfect graphs.* Fulkerson, Knuth, Ravindra. Ehrhard’s TD 2 exercise sheet. Borodulin-Nadzieja–Farkas–Pelczar-Barwacz.

## 10 Conclusion

- Calculus versus coherence space: they are equal at first order and differ from second order on. The difference is exactly the non-sequential behaviours.
- At second order the gap is governed by induced 4-cycles, and it transfers along negative occurrences.
- Negation swaps the global and local calculi, and the coherence space is self-dual between them. The gap at  $T$  and the Bell gap at the core are two sides of one gap.
- The certain-state calculus is not compositional (tight factors, non-tight tensor). The coherence space is compositional by construction, which is what it pays for with extra behaviours.
- Consequence: reasoning in the coherence space certifies weaker guarantees than sequential programs satisfy.
- Open questions: an exact condition beyond second order, a theta-body (quantum) calculus for programs, exponentials beyond `!Bool`, a characterization of the tight formulas, whether  $h$  is quantum (Acín–Fritz–Leverrier–Sainz), and a comparison with Kissinger–Uijlen’s causal processes.

## Data Availability Statement

The Lean 4 development that mechanizes every result marked <sup>Lean</sup> is provided as an anonymized archive with this submission. It builds with `mathlib`, contains no `sorry`, and uses only the axioms `propext`, `Classical.choice`, and `Quot.sound`. It will be made publicly available and submitted for artifact evaluation.

## References

1. mathlib Community, T.: The Lean mathematical library. In: Certified Programs and Proofs (CPP). pp. 367–381. ACM, New York, NY, USA (2020)
2. Danos, V., Ehrhard, T.: Probabilistic coherence spaces as a model of higher-order probabilistic computation. *Information and Computation* **209**(6), 966–991 (2011)
3. de Finetti, B.: *Theory of Probability*, vol. 1. Wiley (1974)
4. Girard, J.Y.: Linear logic. *Theoretical Computer Science* **50**(1), 1–101 (1987)
5. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Conference on Automated Deduction (CADE). pp. 625–635. Springer, Cham, Switzerland (2021)
6. Paris, J.B.: A note on the Dutch book method. In: Proceedings of the Second International Symposium on Imprecise Probabilities and Their Applications (ISIPTA). pp. 301–306 (2001)
7. Williams, J.R.G.: Generalized probabilism: Dutch books and accuracy domination. *Journal of Philosophical Logic* **41**(5), 811–840 (2012)

## A Lean Definitions and Theorems

This appendix lists the Lean 4 statements behind the results marked <sup>Lean</sup>, grouped by section, with proofs omitted. Names are those of the development. The type  $\mathbb{R}_{\geq 0\infty}$  is  $[0, \infty]$ , a formula of Section 3 is a term of the inductive type `Fm`, `web A` is its web, `P A` its probabilistic coherence space, `recipe A` its calculus, and `loc A` its local calculus. Every listed theorem depends only on the axioms `propext`, `Classical.choice`, and `Quot.sound`, except the choice-free results, which avoid `Classical.choice`.

### A.1 The Development

The development builds with `mathlib` and contains no `sorry`. Every listed theorem depends only on the axioms above, which `#print axioms` confirms. The files that contain the results of this paper are the following, with their sizes in lines.

| File                                   | Lines Content                                                  |
|----------------------------------------|----------------------------------------------------------------|
| <code>Recipe</code>                    | 603 presentations, mixtures, affine laws, completeness         |
| <code>Farkas</code>                    | 648 Fourier–Motzkin elimination without choice                 |
| <code>Uniqueness</code>                | 100 uniqueness of the calculus                                 |
| <code>LinearLogicObstruction</code>    | 78 tensor collapse                                             |
| <code>PCSMixture</code>                | 345 first order, data types, $\mathbf{n} \multimap \mathbf{m}$ |
| <code>Coherence</code>                 | 184 certain states and coherence spaces                        |
| <code>Exponential</code>               | 112 !Bool                                                      |
| <code>Linear</code>                    | 384 formulas, their PCS and coherence                          |
| <code>LinearInstances</code>           | 186 concrete results as statements about formulas              |
| <code>Sequential</code>                | 395 sequential programs and the pentagon bound                 |
| <code>Kuhn, GeneralKuhn</code>         | 721 Kuhn’s theorem at the second-order type                    |
| <code>General</code>                   | 389 second order over arbitrary finite data types              |
| <code>ProgLang</code>                  | 235 a language for $T$                                         |
| <code>PCSGraph, PerfectGraph</code>    | 314 STAB, QSTAB, odd holes and antiholes                       |
| <code>ThreeArg</code>                  | 237 a gap among total programs                                 |
| <code>Retract</code>                   | 501 coordinate retracts and the gap at other formulas          |
| <code>Closure</code>                   | 284 closure of the absence of a gap under $\otimes$ and $\&$   |
| <code>CographBasic, CographCore</code> | 1439 forests, cographs, the decomposition                      |
| <code>CographPCS, CographClass</code>  | 629 the exact condition at second order                        |
| <code>ThirdOrder</code>                | 160 a gap without two four-cycles                              |
| <code>TensorGap</code>                 | 81 Shannon’s clique outside the PCS                            |
| <code>LocalCalculus</code>             | 324 local calculi in linear and quantum logic                  |
| <code>Categorical</code>               | 461 duality and tightness (in part)                            |
| <code>Bell</code>                      | 214 the Popescu–Rohrlich box at the core                       |
| <code>Quantum</code>                   | 203 Cabello’s Kochen–Specker set                               |

The general theorems are proved for arbitrary finite data types, and for formulas built from the unit data type with  $\&$  and  $\oplus$  in the exact condition at second order. The finite facts behind the witnesses are checked by evaluation with `decide` on explicit finite types: the five-cycle of the pentagon point, the absence of

three pairwise locally orthogonal events in the support of the Popescu–Rohrlich box, the 64 triples of total functions for the gap among total programs, and the bases of the Kochen–Specker set. The proof of the exact condition at second order uses classical case distinctions on real numbers through the library.

## A.2 Calculi from Certain States

```

def IsSharpVec (x : E → ℝ≥0∞) : Prop := ∀ e, x e = 0 ∨ x e
    = 1

def Mix (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥0∞) :=
  {x | ∃ (n : ℕ) (w : Fin n → ℝ≥0∞) (s : Fin n → E → ℝ≥0∞),
    (∀ i, s i ∈ S) ∧ ∑ i, w i = 1 ∧ x = ∑ i, w i • s i}

theorem sharp_mem_Mix_iff {S : Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S, IsSharpVec s)
  {x : E → ℝ≥0∞} (hx : IsSharpVec x) : x ∈ Mix S ↔ x ∈ S

theorem Mix_Mix (S : Set (E → ℝ≥0∞)) : Mix (Mix S) = Mix S

structure AffineLaw (E : Type*) [Fintype E] where
  a : E → ℝ≥0∞
  b : E → ℝ≥0∞
  k : ℝ≥0∞
  k' : ℝ≥0∞

def AffineLaw.Holds [Fintype E] (L : AffineLaw E) (x : E →
  ℝ≥0∞) : Prop :=
  ∑ e, L.a e * x e + L.k ≤ ∑ e, L.b e * x e + L.k'

theorem AffineLaw.holds_of_Mix [Fintype E] (L : AffineLaw E)
  {S : Set (E → ℝ≥0∞)}
  (hL : ∀ s ∈ S, L.Holds s) {x : E → ℝ≥0∞} (hx : x ∈ Mix
    S) : L.Holds x

theorem Mix_eq_laws' {S : Set (E → ℝ≥0∞)} (hS : ∀ s ∈ S,
  IsSharpVec s) :
  Mix S = {x | ∀ L : AffineLaw E, (∀ s ∈ S, L.Holds s) →
    L.Holds x}

theorem frame_recipe :
  Set.range (fun v : Valuation (LowerSet P) => v.toFun)
    = Mix {f | ∃ v : Valuation (LowerSet P), v.IsSharp ∧
      f = v.toFun}

def Sat {n : Nat} (r : Row n) (w : Fin n → Int) : Prop :=

```

```

0 ≤ dot n r.c w ∧ (r.s = true → 0 < dot n r.c w)

def comb {n : Nat} (p q : Int) (r₁ r₂ : Row n) : Row n :=
  ⟨fun i => p * r₁.c i + q * r₂.c i, (decide (0 < p) && r₁.s) || (decide (0 < q) && r₂.s)⟩

def zeroS (n : Nat) : Row n := ⟨fun _ => 0, true⟩

def lastc (r : Row (n + 1)) : Int := r.c (Fin.last n)

def proj (r : Row (n + 1)) : Row n := ⟨fun i => r.c i.
  castSucc, r.s⟩

def posR (rows : List (Row (n + 1))) := rows.filter fun r
  => decide (0 < lastc r)
def negR (rows : List (Row (n + 1))) := rows.filter fun r
  => decide (lastc r < 0)
def zerR (rows : List (Row (n + 1))) := rows.filter fun r
  => decide (lastc r = 0)

def pair (p q : Row (n + 1)) : Row n := proj (comb (-lastc
  q) (lastc p) p q)

def step (rows : List (Row (n + 1))) : List (Row n) :=
  (zerR rows).map proj ++ (posR rows).flatMap fun p => (
    negR rows).map fun q => pair p q

def extd (D W : Int) (w' : Fin n → Int) : Fin (n + 1) → Int
  :=
  fun i => if h : i.val < n then D * w' ⟨i.val, h⟩ else W

def hrow (μ : Fin N → Int) (ν α : Int) (β : Fin m → Int) :
  Fin (N + 1) → Int :=
  fun i => if h : i.val < N then μ ⟨i.val, h⟩ + α + d * dot
    m β (S ⟨i.val, h⟩)
    else ν - α - dot m β y

def hullRows : List (Row (N + 1)) :=
  (List.finRange N).map (fun j => ⟨hrow S y d (fun k => if
    k = j then 1 else 0) 0 0 (fun _ => 0), false⟩) ++
  [⟨hrow S y d (fun _ => 0) 1 0 (fun _ => 0), true⟩,
   ⟨hrow S y d (fun _ => 0) 0 1 (fun _ => 0), false⟩,
   ⟨hrow S y d (fun _ => 0) 0 (-1) (fun _ => 0), false⟩] ++
  (List.finRange m).flatMap (fun e =>
    [⟨hrow S y d (fun _ => 0) 0 0 (fun k => if k = e then 1
      else 0), false⟩,
     ⟨hrow S y d (fun _ => 0) 0 0 (fun k => if k = e then
      -1 else 0), false⟩])

structure Row (n : Nat) where

```

```

c : Fin n → Int
s : Bool

inductive Der {n : Nat} (rows : List (Row n)) : Row n →
  Prop
| base {r : Row n} : r ∈ rows → Der rows r
| comb {r1 r2 : Row n} (p q : Int) : 0 ≤ p → 0 ≤ q → Der
  rows r1 → Der rows r2 →
  Der rows (comb p q r1 r2)

theorem fm : ∀ (n : Nat) (rows : List (Row n)), (∃ w, ∀ r ∈
  rows, Sat r w) ∨ Der rows (zeroS n)

theorem complete (hd : 0 < d)
  (hlaw : ∀ (c : Fin m → Int) (k : Int), (∀ j, k ≤ dot m
    c (S j)) → k * d ≤ dot m c y) :
  ∃ (lam : Fin N → Int) (t : Int), 0 < t ∧ (∀ j, 0 ≤ lam
    j) ∧
    dot N (fun _ => 1) lam = t ∧ ∀ e, d * dot N lam (fun
      j => S j e) = t * y e

theorem sound (hd : 0 < d) (lam : Fin N → Int) (t : Int) (
  ht : 0 < t) (hl : ∀ j, 0 ≤ lam j)
  (hsum : dot N (fun _ => 1) lam = t) (hx : ∀ e, d * dot
    N lam (fun j => S j e) = t * y e)
  (c : Fin m → Int) (k : Int) (hc : ∀ j, k ≤ dot m c (S j
    )) : k * d ≤ dot m c y

theorem calculus_unique {S C : Set (E → ℝ≥0∞)} (hS : ∀ s ∈
  S, IsSharpVec s) (hSC : S ⊆ C)
  (hconv : Mix C ⊆ C)
  (hlaws : ∀ L : AffineLaw E, (∀ s ∈ S, L.Holds s) → ∀ x
    ∈ C, L.Holds x) :
  C = Mix S

theorem calculus_laws_iff {S C : Set (E → ℝ≥0∞)} (hSC : S ⊆
  C)
  (hlaws : ∀ L : AffineLaw E, (∀ s ∈ S, L.Holds s) → ∀ x
    ∈ C, L.Holds x) (L : AffineLaw E) :
  (∀ x ∈ C, L.Holds x) ↔ ∀ s ∈ S, L.Holds s

def IsTensorValuation (v : Ω → ℝ≥0∞) (t : Ω → Ω → Ω) : Prop
:=
  ∀ a b, v a + v b = v (a ⊔ b) + v (t a b)

```

Theorem 4.

```

theorem IsTensorValuation.collapse {v : Ω → ℝ≥0∞} {t : Ω →
  Ω → Ω}
  (h : IsTensorValuation v t) {a : Ω} (ha : v a ≠ ⊤) :

```

$$v (t \ a \ a) = v \ a$$

Example 1.

```
theorem forced_constant_of_add_valuation {v : ℕ → ℝ≥0∞}
  (h : IsTensorValuation v (· + ·)) (hfin : ∀ n, v n ≠ ⊤)
  (n : ℕ) :
  v n = v 0
```

Theorem 4, second form.

```
theorem not_isTensorValuation_of_discriminates {v : Ω → ℝ≥0∞}
  {t : Ω → Ω → Ω} {a : Ω} (ha : v a ≠ ⊤) (hne : v (t a a)
    ≠ v a) :
  ¬ IsTensorValuation v t
```

### A.3 Linear Logic

```
def flat (X : Type*) [Fintype X] : Set (X → ℝ≥0∞) := {x | ∑
  a, x a ≤ 1}

noncomputable def app (t : X × Y → ℝ≥0∞) (x : X → ℝ≥0∞) : Y
  → ℝ≥0∞ :=
  fun b => ∑ a, x a * t (a, b)

def lin (X Y : Type*) [Fintype X] [Fintype Y] : Set (X × Y
  → ℝ≥0∞) :=
  {t | ∀ x ∈ flat X, app t x ∈ flat Y}

theorem mem_lin_iff [DecidableEq X] (t : X × Y → ℝ≥0∞) :
  t ∈ lin X Y ↔ ∀ a, ∑ b, t (a, b) ≤ 1

noncomputable def pairing (x y : E → ℝ≥0∞) : ℝ≥0∞ := ∑ e, x
  e * y e

def row (a : X) : X × Y → ℝ≥0∞ := fun ab => if ab.1 = a
  then 1 else 0

def orth (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥0∞) := {y | ∀ x
  ∈ S, pairing x y ≤ 1}

structure IsPCS (P : Set (E → ℝ≥0∞)) : Prop where
  closed : orth (orth P) = P
  total : ∀ e, ∃ x ∈ P, x e ≠ 0
  bounded : ∀ e, ∃ M ≠ ⊤, ∀ x ∈ P, x e ≤ M

theorem isPCS_lin : IsPCS (lin X Y)

theorem isPCS_flat : IsPCS (flat X)
```

```

theorem isPCS_tensDual : IsPCS tensDual

theorem pcs_recipe {X Y : Type} [Fintype X] [Fintype Y] [
  DecidableEq X] :
  lin X Y = Mix (Set.range (det : (X → Option Y) → X × Y
    →  $\mathbb{R}_{\geq 0\infty}$ ))

def flatC (X : Type*) : X → X → Prop := fun x y => x = y

def dualC {X : Type*} (c : X → X → Prop) : X → X → Prop :=
  fun x y => x = y ∨ ¬ c x y

def tensC {X Y : Type*} (c : X → X → Prop) (d : Y → Y →
  Prop) : X × Y → X × Y → Prop :=
  fun p q => c p.1 q.1 ∧ d p.2 q.2

def arrowC {X Y : Type*} (c : X → X → Prop) (d : Y → Y →
  Prop) : X × Y → X × Y → Prop :=
  dualC (tensC c (dualC d))

def IsClique {X : Type*} (c : X → X → Prop) (B : X → Prop)
  : Prop := ∀ x y, B x → B y → c x y

def secondC : W × W → W × W → Prop :=
  dualC (tensC (arrowC (flatC Bool) (flatC Bool)) (arrowC (
    flatC Bool) (flatC Bool)))

noncomputable def throughAll (C : Finset W) : Bool → Option
  Bool := fun a =>
  if h : ∃ p ∈ C, p.1 = a then some (Classical.choose h).2
  else none

theorem sharp_lin_iff_clique {X Y : Type*} [Fintype X] [
  Fintype Y] [DecidableEq X] [DecidableEq Y]
  {t : X × Y →  $\mathbb{R}_{\geq 0\infty}$ } (h01 : ∀ p, t p = 0 ∨ t p = 1) :
  t ∈ lin X Y ↔ IsClique (arrowC (flatC X) (flatC Y)) (t
    . = 1)

inductive Fm where
| base (n : ℕ)
| neg (A : Fm)
| tens (A B : Fm)
| wth (A B : Fm)

def web : Fm → Type
| base n => Fin n
| neg A => web A
| tens A B => web A × web B

```

```

| wth A B => web A  $\oplus$  web B

def P : (A : Fm)  $\rightarrow$  Set (web A  $\rightarrow \mathbb{R}_{\geq 0\infty}$ )
| base n => flat (Fin n)
| neg A => orth (P A)
| tens A B => orth (orth {z |  $\exists x \in P A, \exists y \in P B, z =$ 
    tvec x y})
| wth A B => {z | (fun a => z (Sum.inl a))  $\in P A \wedge$  (fun b
    => z (Sum.inr b))  $\in P B$ }

def coh : (A : Fm)  $\rightarrow$  web A  $\rightarrow$  web A  $\rightarrow$  Prop
| base _ => fun x y => x = y
| neg A => fun x y => x = y  $\vee \neg$  coh A x y
| tens A B => fun s t => coh A s.1 t.1  $\wedge$  coh B s.2 t.2
| wth A B => fun s t => match s, t with
| Sum.inl a, Sum.inl a' => coh A a a'
| Sum.inr b, Sum.inr b' => coh B b b'
| _, _ => True

theorem isPCS_P :  $\forall A : Fm, \text{IsPCS } (P A)$ 

def par (A B : Fm) : Fm := neg (tens (neg A) (neg B))
def plus (A B : Fm) : Fm := neg (wth (neg A) (neg B))
def lolli (A B : Fm) : Fm := neg (tens A (neg B))

noncomputable def tvec {X Y : Type} (x : X  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (y : Y  $\rightarrow$ 
 $\mathbb{R}_{\geq 0\infty}$ ) : X  $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
fun ab => x ab.1 * y ab.2

noncomputable def svec {X Y : Type} (x : X  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (y : Y  $\rightarrow$ 
 $\mathbb{R}_{\geq 0\infty}$ ) : X  $\oplus$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
| Sum.inl a => x a
| Sum.inr b => y b

noncomputable def pr (s t : E) : E  $\rightarrow \mathbb{R}_{\geq 0\infty}$  := Pi.single s 1
+ Pi.single t 1

theorem invariants :  $\forall A : Fm,$ 
( $\forall s : \text{web } A, \text{Pi.single } s \ 1 \in P A \wedge \forall x \in P A, x \ s \leq 1$ )
 $\wedge$ 
( $\forall s \ t : \text{web } A, s \neq t \rightarrow \text{coh } A \ s \ t \rightarrow \text{pr } s \ t \in P A$ )  $\wedge$ 
( $\forall s \ t : \text{web } A, s \neq t \rightarrow \neg \text{coh } A \ s \ t \rightarrow \forall x \in P A, x \ s +$ 
 $x \ t \leq 1$ )

theorem single_mem_P (A : Fm) (s : web A) : Pi.single s 1  $\in$ 
P A

theorem pair_mem_P (A : Fm) {s t : web A} (hst : s  $\neq$  t) (h
: coh A s t) : pr s t  $\in P A$ 

```

```

theorem sharp_isClique (A : Fm) {z : web A → ℝ≥0∞} (hz : z
  ∈ P A)
  (h01 : ∀ s, z s = 0 ∨ z s = 1) : ∀ s t, z s = 1 → z t =
    1 → coh A s t

theorem recipe_subset_P (A : Fm) :
  Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s = 1} ⊆ P A

theorem recipe_sharp_iff (A : Fm) {x : web A → ℝ≥0∞} (hx :
  ∀ s, x s = 0 ∨ x s = 1) :
  x ∈ Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s = 1} ↔ x ∈ P
    A

theorem Mix_subset_orth {S T : Set (E → ℝ≥0∞)} (h : S ⊆
  orth T) : Mix S ⊆ orth T

theorem P_lolli_base (n m : ℕ) : P (lolli (base n) (base m)
  ) = lin (Fin n) (Fin m)

def boolFin {k : ℕ} (hk : 2 ≤ k) : Bool → Fin k

theorem recipe_first_order (n m : ℕ) :
  P (lolli (base n) (base m)) = ConstructiveProb.Mix (Set
    .range
    (det : (Fin n → Option (Fin m)) → Fin n × Fin m → ℝ≥0
      ∞))

theorem recipe_second_order {n m n' m' : ℕ} (hn : 0 < n) (
  hn' : 0 < n') :
  ConstructiveProb.Mix {z | z ∈ P (second n m n' m') ∧ ∀
    u, z u = 0 ∨ z u = 1}
    = sequentialG (Fin n) (Fin m) (Fin n') (Fin m')

theorem recipe_second_order_ssubset {n m n' m' : ℕ} (hn : 2
  ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n')
  (hm' : 2 ≤ m') :
  ConstructiveProb.Mix {z | z ∈ P (second n m n' m') ∧ ∀
    u, z u = 0 ∨ z u = 1}
    ⊆ P (second n m n' m')

theorem sharp_bang_mixed_zero {z : ℕ × ℕ → ℝ≥0∞} (hz : z ∈
  bang) (h01 : ∀ m, z m = 0 ∨ z m = 1)
  {i j : ℕ} (hi : 1 ≤ i) (hj : 1 ≤ j) : z (i, j) = 0

def orthI {E : Type*} (S : Set (E → ℝ≥0∞)) : Set (E → ℝ≥0∞)
  := {y | ∀ x ∈ S, ∑' e, x e * y e ≤ 1}

def gens : Set (ℕ × ℕ → ℝ≥0∞) := {u | ∃ x ∈ flat Bool, u =
  prom x}

```

```

noncomputable def half : Bool → ℝ≥0∞ := fun _ => 2-1

noncomputable def prom (x : Bool → ℝ≥0∞) : ℕ × ℕ → ℝ≥0∞ :=
  fun m => x true ^ m.1 * x false ^ m.2

def bang : Set (ℕ × ℕ → ℝ≥0∞) := orthI (orthI gens)

theorem prom_half_not_mix : prom half ∈ bang ∧
  prom half ∉ Mix {z | z ∈ bang ∧ ∀ m, z m = 0 ∨ z m = 1}

```

#### A.4 Second Order

```

def cov (f : X → Option Y) (g : X' → Option Y') (pq : (X ×
  Y) × (X' × Y')) : Prop :=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some pq.2.2

def cp {X Y : Type} (p q : X × Y) : Prop := p = q ∨ p.1 ≠ q
  .1

def cp2 (s t : (X × Y) × (X' × Y')) : Prop := cp s.1 t.1 ∧
  cp s.2 t.2

noncomputable def thru {X Y : Type} [DecidableEq X] (C :
  Finset (X × Y)) : X → Option Y :=
  fun a => if h : ∃ p ∈ C, p.1 = a then some (Classical.
    choose h).2 else none

def GTree (c : X') (af : Y' → X) (pq : (X × Y) × (X' × Y'))
  : Prop :=
  pq.2.1 = c ∧ pq.1.1 = af pq.2.2

def tD (X Y X' Y' : Type) [Fintype X] [Fintype Y] [Fintype
  X'] [Fintype Y'] :
  Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
  {t | ∀ y ∈ lin X Y, ∀ z ∈ lin X' Y', ∑ p, ∑ q, y p * t (p
    , q) * z q ≤ 1}

def FTree (a : X) (cf : Y → X') (pq : (X × Y) × (X' × Y'))
  : Prop :=
  pq.1.1 = a ∧ pq.2.1 = cf pq.1.2

theorem sharp_tD_iff [Nonempty X] [Nonempty X'] {z : (X × Y
  ) × (X' × Y') → ℝ≥0∞}
  (h01 : ∀ pq, z pq = 0 ∨ z pq = 1) :
  z ∈ tD X Y X' Y' ↔
  (∃ a cf, ∀ pq, z pq = 1 → FTree a cf pq) ∨ (∃ c af, ∀
    pq, z pq = 1 → GTree c af pq)

```

```

noncomputable def Core.val (K : Core A B C D) (a : A) (b :
  B) (c : C) (d : D) :  $\mathbb{R}_{\geq 0\infty}$  :=
  K. $\mu$  a * K. $\kappa$  a b c * K. $\rho$  a b c d

noncomputable def Core.mass (K : Core A B C D) :  $\mathbb{R}_{\geq 0\infty}$  :=  $\sum$ 
  a, K. $\mu$  a

theorem sharp_mem_sequentialG [Nonempty X] [Nonempty X'] {z
  : (X  $\times$  Y)  $\times$  (X'  $\times$  Y')  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ }
  (hz : z  $\in$  tD X Y X' Y') (h01 :  $\forall$  u, z u = 0  $\vee$  z u = 1)
  : z  $\in$  sequentialG X Y X' Y'

structure Core (A B C D : Type) [Fintype A] [Fintype C]
  where
     $\mu$  : A  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
     $\kappa$  : A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
     $\rho$  : A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$  D  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
    h $\kappa$  :  $\forall$  a b,  $\sum$  c,  $\kappa$  a b c  $\leq$  1
    h $\rho$  :  $\forall$  a b c d,  $\rho$  a b c d  $\leq$  1

def sequentialG (X Y X' Y' : Type) [Fintype X] [Fintype X']
  : Set ((X  $\times$  Y)  $\times$  (X'  $\times$  Y'))  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  :=
  {t |  $\exists$  (F : Core X Y X' Y') (G : Core X' Y' X Y), F.mass
    + G.mass  $\leq$  1  $\wedge$ 
    t = fun pq => F.val pq.1.1 pq.1.2 pq.2.1 pq.2.2 + G.val
      pq.2.1 pq.2.2 pq.1.1 pq.1.2}

theorem recipe_eq_sequentialG [Nonempty X] [Nonempty X'] :
  Mix {z | z  $\in$  tD X Y X' Y'  $\wedge$   $\forall$  u, z u = 0  $\vee$  z u = 1} =
  sequentialG X Y X' Y'

def sharpSeqG (X Y X' Y' : Type) [Fintype X] [Fintype X'] :
  Set ((X  $\times$  Y)  $\times$  (X'  $\times$  Y'))  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  :=
  {z | z  $\in$  sequentialG X Y X' Y'  $\wedge$   $\forall$  u, z u = 0  $\vee$  z u = 1}

theorem sequentialG_subset_Mix : sequentialG X Y X' Y'  $\subseteq$ 
  Mix (sharpSeqG X Y X' Y')

noncomputable def compl' (m : A  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) : Option A  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
  | none => 1 -  $\sum$  a, m a
  | some a => m a

abbrev ChoiceG (A B C D : Type) := (Unit  $\rightarrow$  Option A)  $\times$  (A  $\times$ 
  B  $\rightarrow$  Option C)  $\times$  (A  $\times$  B  $\times$  C  $\times$  D  $\rightarrow$  Bool)

noncomputable def detCoreG [DecidableEq A] [DecidableEq C]
  ( $\sigma$  : ChoiceG A B C D) : Core A B C D

```

```

theorem Mix_sequentialG : Mix (sequentialG X Y X' Y') =
  sequentialG X Y X' Y'

noncomputable def comp (m : Bool → ℝ≥0∞) : Option Bool → ℝ≥0∞
  | none => 1 - (m true + m false)
  | some b => m b

noncomputable def bern (r : ℝ≥0∞) : Bool → ℝ≥0∞
  | true => r
  | false => 1 - r

abbrev Choice := (Unit → Option Bool) × (Bool × Bool →
  Option Bool) × (Bool × Bool × Bool × Bool → Bool)

noncomputable def detCore (σ : Choice) (a b c d : Bool) : ℝ≥0∞ :=
  (if σ.1 () = some a then 1 else 0) * (if σ.2.1 (a, b) =
    some c then 1 else 0) *
  (if σ.2.2 (a, b, c, d) = true then 1 else 0)

noncomputable def detF (σ : Choice) : FFirst where
  μ a

noncomputable def detG (σ : Choice) : GFirst where
  μ c

def sharpSeq : Set (W × W → ℝ≥0∞) := {z | z ∈ sequential ∧
  ∀ pq, z pq = 0 ∨ z pq = 1}

structure Beh where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hκ : ∀ a b, κ a b true + κ a b false ≤ 1
  hρ : ∀ a b c d, ρ a b c d ≤ 1

noncomputable def Beh.val (B : Beh) (a b c d : Bool) : ℝ≥0∞
  := B.μ a * B.κ a b c * B.ρ a b c d

noncomputable def Beh.mass (B : Beh) : ℝ≥0∞ := B.μ true + B
  .μ false

def seqU : Set (W × W → ℝ≥0∞) :=
  {t | ∃ BF BG : Beh, BF.mass + BG.mass ≤ 1 ∧
    t = fun pq => BF.val pq.1.1 pq.1.2 pq.2.1 pq.2.2 + BG.
      val pq.2.1 pq.2.2 pq.1.1 pq.1.2}

```

```

theorem FFirst.den_mem_Mix (F : FFirst) : F.den ∈ Mix
  sharpSeq

theorem sequential_subset_Mix : sequential ⊆ Mix sharpSeq

theorem Mix_sequential : Mix sequential = sequential

theorem recipe_second_order_eq_sequential' :
  Mix {z | z ∈ tensDual ∧ ∀ pq, z pq = 0 ∨ z pq = 1} =
    sequential

inductive PEnd
| halt
| diverge
| choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
  (p : Fin k → PEnd)

inductive POne (C D : Type)
| diverge
| choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
  (p : Fin k → POne C D)
| call (c : C) (k : D → PEnd)

inductive P0 (X Y X' Y' : Type)
| diverge
| choose (k : Nat) (w : Fin k → ℝ≥0∞) (hw : ∑ i, w i ≤ 1)
  (p : Fin k → P0 X Y X' Y')
| callF (a : X) (k : Y → POne X' Y')
| callG (c : X') (k : Y' → POne X Y)

noncomputable def PEnd.den : PEnd → ℝ≥0∞
| .halt => 1
| .diverge => 0
| .choose _ w _ p => ∑ i, w i * (p i).den

noncomputable def POne.den {C D : Type} [DecidableEq C] :
  POne C D → C × D → ℝ≥0∞
| .diverge => fun _ => 0
| .choose _ w _ p => fun cd => ∑ i, w i * (p i).den cd
| .call c k => fun cd => if cd.1 = c then (k cd.2).den
  else 0

noncomputable def P0.den : P0 X Y X' Y' → (X × Y) × (X' × Y
  ') → ℝ≥0∞
| .diverge => fun _ => 0
| .choose _ w _ p => fun u => ∑ i, w i * (p i).den u
| .callF a k => fun u => if u.1.1 = a then (k u.1.2).den
  u.2 else 0

```

```

| .callG c k => fun u => if u.2.1 = c then (k u.2.2).den
  u.1 else 0

theorem P0.sound :  $\forall p : P0\ X\ Y\ X'\ Y', p.den \in \text{sequentialG}$ 
  X Y X' Y'

theorem P0.den_mem_recipe [Nonempty X] [Nonempty X'] (p :
  P0 X Y X' Y') :
  p.den  $\in \text{Mix } \{z \mid z \in \text{tD } X\ Y\ X'\ Y' \wedge \forall u, z\ u = 0 \vee z\ u = 1\}$ 

def PEnd.ofBool (b : Bool) : PEnd := if b then .halt else .
  diverge

theorem P0.sharp_definable [Nonempty X] [Nonempty X'] {z :
  (X  $\times$  Y)  $\times$  (X'  $\times$  Y')}  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
  (hz : z  $\in \text{tD } X\ Y\ X'\ Y'$ ) (h01 :  $\forall u, z\ u = 0 \vee z\ u = 1$ )
  :  $\exists p : P0\ X\ Y\ X'\ Y', p.den = z$ 

theorem recipe_eq_denotations [Nonempty X] [Nonempty X'] :
  Mix {z | z  $\in \text{tD } X\ Y\ X'\ Y' \wedge \forall u, z\ u = 0 \vee z\ u = 1$ } =
  Set.range (P0.den (X := X) (Y := Y)
    (X' := X') (Y' := Y'))

def emb (s : W  $\times$  W) : (X  $\times$  Y)  $\times$  (X'  $\times$  Y') :=
  ((ex s.1.1, ey s.1.2), (ex' s.2.1, ey' s.2.2))

noncomputable def pentE (ex : Bool  $\rightarrow$  X) (ey : Bool  $\rightarrow$  Y) (ex'
  : Bool  $\rightarrow$  X') (ey' : Bool  $\rightarrow$  Y') :
  (X  $\times$  Y)  $\times$  (X'  $\times$  Y')  $\rightarrow \mathbb{R}_{\geq 0\infty} :=$ 
  fun u => if u  $\in \text{pent.image (emb ex ey ex' ey')}$  then  $2^{-1}$ 
    else 0

theorem pentE_not_mix (hx : Function.Injective ex) (hy :
  Function.Injective ey)
  (hx' : Function.Injective ex') (hy' : Function.
    Injective ey')
  {n :  $\mathbb{N}$ } (w : Fin n  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hw :  $\sum i, w\ i = 1$ ) (z : Fin
    n  $\rightarrow (X \times Y) \times (X' \times Y') \rightarrow \mathbb{R}_{\geq 0\infty}$ )
  (hz :  $\forall i, z\ i \in \text{tD } X\ Y\ X'\ Y'$ ) (h01 :  $\forall i\ u, z\ i\ u = 0$ 
     $\vee z\ i\ u = 1$ ) :
  pentE ex ey ex' ey'  $\neq \sum i, w\ i \bullet z\ i$ 

theorem pentE_mem (hx : Function.Injective ex) (hy :
  Function.Injective ey)
  (hx' : Function.Injective ex') (hy' : Function.
    Injective ey') :
  pentE ex ey ex' ey'  $\in \text{tD } X\ Y\ X'\ Y'$ 

```

```

noncomputable def idBool : Bool × Bool →  $\mathbb{R}_{\geq 0\infty}$  := fun ab =>
  if ab.1 = ab.2 then 1 else 0

noncomputable def det (f : X → Option Y) : X × Y →  $\mathbb{R}_{\geq 0\infty}$ 

noncomputable def rowDist (t : X × Y →  $\mathbb{R}_{\geq 0\infty}$ ) (a : X) :
  Option Y →  $\mathbb{R}_{\geq 0\infty}$ 
| none => 1 -  $\sum b, t(a, b)$ 
| some b => t(a, b)

noncomputable def tens (x : X →  $\mathbb{R}_{\geq 0\infty}$ ) (y : Y →  $\mathbb{R}_{\geq 0\infty}$ ) : X ×
  Y →  $\mathbb{R}_{\geq 0\infty}$  := fun ab => x ab.1 * y ab.2

abbrev W := Bool × Bool

def pent : Finset (W × W) :=
  {((true, true), (true, true)), ((true, false), (false,
    false)),
    ((false, true), (true, false)), ((false, true), (false,
    true)),
    ((false, false), (false, false))}

noncomputable def pentElt : W × W →  $\mathbb{R}_{\geq 0\infty}$  := fun pq => if pq
  ∈ pent then  $2^{-1}$  else 0

def covers (f g : Bool → Option Bool) (pq : W × W) : Prop
:=
  f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some pq.2.2

def compat (p q : W) : Prop := p = q ∨ p.1 ≠ q.1

def compat2 (s t : W × W) : Prop := compat s.1 t.1 ∧ compat
  s.2 t.2

def through (p q : W) : Bool → Option Bool := fun b => if b
  = p.1 then some p.2 else some q.2

theorem pent_clique_le_two : ∀ B ∈ pent.powerset,
  (∀ s ∈ B, ∀ t ∈ B, compat2 s t) → B.card ≤ 2

theorem pent_indep_le_two : ∀ B ∈ pent.powerset,
  (∀ s ∈ B, ∀ t ∈ B, compat2 s t → s = t) → B.card ≤ 2

structure FFirst where
  μ : Bool →  $\mathbb{R}_{\geq 0\infty}$ 
  κ : Bool → Bool → Bool →  $\mathbb{R}_{\geq 0\infty}$ 
  ρ : Bool → Bool → Bool → Bool →  $\mathbb{R}_{\geq 0\infty}$ 
  hμ : μ true + μ false ≤ 1
  hκ : ∀ a b, κ a b true + κ a b false ≤ 1

```

```

hρ : ∀ a b c d, ρ a b c d ≤ 1

structure GFirst where
  μ : Bool → ℝ≥0∞
  κ : Bool → Bool → Bool → ℝ≥0∞
  ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
  hμ : μ true + μ false ≤ 1
  hκ : ∀ c d, κ c d true + κ c d false ≤ 1
  hρ : ∀ c d a b, ρ c d a b ≤ 1

noncomputable def FFirst.den (S : FFirst) : W × W → ℝ≥0∞ :=
  fun pq =>
    S.μ pq.1.1 * S.κ pq.1.1 pq.1.2 pq.2.1 * S.ρ pq.1.1 pq.1.2
      pq.2.1 pq.2.2

noncomputable def GFirst.den (S : GFirst) : W × W → ℝ≥0∞ :=
  fun pq =>
    S.μ pq.2.1 * S.κ pq.2.1 pq.2.2 pq.1.1 * S.ρ pq.2.1 pq.2.2
      pq.1.1 pq.1.2

noncomputable def copyStrategy : FFirst where
  μ a

def idleG : GFirst where
  μ -

def idleF : FFirst where
  μ -

def tensDual : Set (W × W → ℝ≥0∞) :=
  {t | ∀ y ∈ lin Bool Bool, ∀ z ∈ lin Bool Bool, ∑ p, ∑ q,
    y p * t (p, q) * z q ≤ 1}

def sequential : Set (W × W → ℝ≥0∞) :=
  {t | ∃ (α β : ℝ≥0∞) (F : FFirst) (G : GFirst), α + β ≤ 1
    ∧ t = α • F.den + β • G.den}

theorem seq_pent_le_two {t : W × W → ℝ≥0∞} (ht : t ∈
  sequential) : ∑ pq ∈ pent, t pq ≤ 2

theorem seq_pent_tight : ∃ t ∈ sequential, ∑ pq ∈ pent, t
  pq = 2

theorem pentElt_pent_sum : ∑-1 pq ∈ pent, pentElt pq = 5 * 2

theorem tensDual_pent_le {t : W × W → ℝ≥0∞} (ht : t ∈
  tensDual) :
  2 * ∑ pq ∈ pent, t pq ≤ 5
    
```

```

def second (n m n' m' : ℕ) : Fm := neg (tens (lolli (base n
  ) (base m)) (lolli (base n') (base m'))))

theorem P_second (n m n' m' : ℕ) :
  P (second n m n' m') = tD (Fin n) (Fin m) (Fin n') (Fin
    m')

abbrev L2 : Fm := lolli (base 2) (base 2)

abbrev S3 : Fm := tens (tens L2 L2) L2

abbrev T3 : Fm := Fm.neg S3

noncomputable def graph2 (f : Fin 2 → Fin 2) : web L2 → ℝ≥0
  ∞ := fun p => if f p.1 = p.2 then 1 else 0

theorem graph2_mem (f : Fin 2 → Fin 2) : graph2 f ∈ P L2

theorem sharp_L2_le {s : web L2 → ℝ≥0∞} (hs : s ∈ sharpP L2
  ) : ∃ f, s ≤ graph2 f

theorem sharp_L2L2_le {s : web (tens L2 L2) → ℝ≥0∞} (hs : s
  ∈ sharpP (tens L2 L2)) :
  ∃ f g, s ≤ tvec (graph2 f) (graph2 g)

def k3 (v : W3) : ℕ := if v = onePt then 2 else if v ∈
  halfPts then 1 else 0

noncomputable def y5 : web T3 → ℝ≥0∞ := fun v => (k3 v : ℝ≥0
  ∞) * 2-1

theorem y5_total (f g h : Fin 2 → Fin 2) :
  pairing (tvec (tvec (graph2 f) (graph2 g)) (graph2 h))
    y5 = 1

theorem y5_mem : y5 ∈ P T3

theorem sharp5_le {s : web T3 → ℝ≥0∞} (hs : s ∈ P T3) (h01
  : ∀ v, s v = 0 ∨ s v = 1) :
  s c0 + s c1 + s c2 + s c3 + s c4 ≤ 2

theorem gap_T3 : recipe T3 ⊂ P T3

```

## A.5 The Exact Condition

```

def excl (s t : (X × Y) × (X' × Y')) : Prop := s ≠ t ∧ cp2
  s t

```

```

theorem tD_eq_QSTAB : tD X Y X' Y' = QSTAB (excl (X := X) (
  Y := Y) (X' := X') (Y' := Y'))

theorem recipe_eq_STAB :
  Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u = 1}
    = STAB (excl (X := X) (Y := Y) (X' := X') (Y' := Y'))

def cyc {n : ℕ} (i j : ZMod n) : Prop := j = i + 1 ∨ i = j
  + 1

def IsCliqueG (C : Finset V) : Prop := ∀ u ∈ C, ∀ v ∈ C, u
  ≠ v → adj u v

def IsStableG (I : Finset V) : Prop := ∀ u ∈ I, ∀ v ∈ I, ¬
  adj u v

noncomputable def ind (I : Finset V) : V → ℝ≥0∞ := fun v =>
  if v ∈ I then 1 else 0

structure Antihole (adj : V → V → Prop) (n : ℕ) where
  h : ZMod n → V
  inj : Function.Injective h
  adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j) ↔ ¬ cyc i j)

def QSTAB : Set (V → ℝ≥0∞) := {x | ∀ C, IsCliqueG adj C → ∑
  v ∈ C, x v ≤ 1}

def STAB : Set (V → ℝ≥0∞) := Mix {x | ∃ I, IsStableG adj I
  ∧ x = ind I}

structure Hole (adj : V → V → Prop) (n : ℕ) where
  h : ZMod n → V
  inj : Function.Injective h
  adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j) ↔ cyc i j)

theorem berge_of_stab_eq_qstab (heq : STAB adj = QSTAB adj)
  (k : ℕ) (hk : 2 ≤ k)
  [NeZero (2 * k + 1)] : IsEmpty (Hole adj (2 * k + 1)) ∧
  IsEmpty (Antihole adj (2 * k + 1))

theorem berge_of_recipe_eq_pcs
  (heq : Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u = 0 ∨ z u =
    1} = tD X Y X' Y')
  (k : ℕ) (hk : 2 ≤ k) [NeZero (2 * k + 1)] :
  IsEmpty (Hole (excl (X := X) (Y := Y) (X' := X') (Y' :=
    Y')) (2 * k + 1)) ∧
  IsEmpty (Antihole (excl (X := X) (Y := Y) (X' := X')
    (Y' := Y')) (2 * k + 1))

```

```

theorem orth_orth_subset_Mix {S : Set (E → ℝ≥0∞)} (hSf : S.
  Finite)
  (hS : ∀ s ∈ S, ConstructiveProb.IsSharpVec s) (h0 : (0
    : E → ℝ≥0∞) ∈ S)
  (hdown : ∀ s ∈ S, ∀ t : E → ℝ≥0∞, ConstructiveProb.
    IsSharpVec t → t ≤ s → t ∈ S) :
  orth (orth S) ⊆ ConstructiveProb.Mix S

theorem orth_Mix (S : Set (E → ℝ≥0∞)) : orth (
  ConstructiveProb.Mix S) = orth S

theorem recipe_orth_orth (A : Fm) : orth (orth (recipe A))
  = recipe A

theorem recipe_tens {A B : Fm} (hA : recipe A = P A) (hB :
  recipe B = P B) :
  recipe (tens A B) = P (tens A B)

theorem recipe_wth {A B : Fm} (hA : recipe A = P A) (hB :
  recipe B = P B) :
  recipe (wth A B) = P (wth A B)

def sharpP (A : Fm) : Set (web A → ℝ≥0∞) := {z | z ∈ P A ∧
  ∀ s, z s = 0 ∨ z s = 1}

theorem P_neg_tens_eq {A B : Fm} (hA : recipe A = P A) (hB
  : recipe B = P B) :
  P (Fm.neg (tens A B)) = orth {z | ∃ s ∈ sharpP A, ∃ t ∈
    sharpP B, z = tvec s t}

def svec' {X Y : Type} (x : X → ℝ≥0∞) (y : Y → ℝ≥0∞) : X ⊕
  Y → ℝ≥0∞
| Sum.inl a => x a
| Sum.inr b => y b

theorem tvec_sum {X Y : Type} [Fintype X] [Fintype Y] {m n
  : ℕ} (w : Fin m → ℝ≥0∞)
  (s : Fin m → X → ℝ≥0∞) (v : Fin n → ℝ≥0∞) (t : Fin n →
    Y → ℝ≥0∞) :
  tvec (∑ i, w i • s i) (∑ j, v j • t j)
  = ∑ ij : Fin m × Fin n, (w ij.1 * v ij.2) • tvec (s
    ij.1) (t ij.2)

theorem recipe_lolli_base (n m : ℕ) :
  recipe (lolli (base n) (base m)) = P (lolli (base n) (
    base m))

theorem recipe_core (n m n' m' : ℕ) : recipe (core n m n' m
  ') = P (core n m n' m')

```

```

inductive ACode
| one
| with (a b : ACode)
| plus (a b : ACode)

def toFm : ACode → Fm
| one => Fm.base 1
| with a b => Fm.with a.toFm b.toFm
| plus a b => Fm.plus a.toFm b.toFm

abbrev W (a : ACode) : Type := web a.toFm

theorem recipe_plus {A B : Fm} (hA : recipe A = P A) (hB :
  recipe B = P B) :
  recipe (Fm.plus A B) = P (Fm.plus A B)

theorem recipe_code : ∀ a : ACode, recipe a.toFm = P a.toFm

theorem clique_mem_P : ∀ (a : ACode) (c : W a →  $\mathbb{R}_{\geq 0\infty}$ ), (∀ x
  , c x = 0 ∨ c x = 1) →
  (∀ x y, c x = 1 → c y = 1 → ACode.coh a x y) → c ∈ P a.
  toFm

inductive TCode
| one
| plus (s t : TCode)
| cone (t : TCode)

def toA : TCode → ACode
| one => ACode.one
| plus s t => ACode.plus s.toA t.toA
| cone t => ACode.with t.toA ACode.one

structure GIso (a b : ACode) where
  e : W a ≅ W b
  coh_iff : ∀ x y, ACode.coh a x y ↔ ACode.coh b (e x) (e y)

def GIso.retract {a b : ACode} (I : GIso a b) : Retract a.
  toFm b.toFm

def retractSwap (A B : Fm) : Retract (Fm.tens A B) (Fm.tens
  B A)

def HasC4 (a : ACode) : Prop :=
  ∃ p q r s : W a, ACode.coh a p q ∧ ACode.coh a q r ∧
    ACode.coh a r s ∧ ACode.coh a s p ∧
    ¬ ACode.coh a p r ∧ ¬ ACode.coh a q s

```

```

theorem hasC4_iff (a : ACode) : HasC4 a  $\leftrightarrow$  Nonempty (C4 a.
  toFm)

def Complete (a : ACode) : Prop :=  $\forall$  x y, ACode.coh a x y

theorem complete_or {a b : ACode} (h :  $\neg$  HasC4 (ACode.with a
  b)) : Complete a  $\vee$  Complete b

theorem tree_of_noC4 :  $\forall$  a : ACode,  $\neg$  HasC4 a  $\rightarrow$   $\exists$  t : TCode
  , Nonempty (GIso a t.toA)

noncomputable def  $\omega$  : (a : ACode)  $\rightarrow$  (W a  $\rightarrow \mathbb{R}$ )  $\rightarrow \mathbb{R}$ 
| one, v => v pt
| with a b, v =>  $\omega$  a (fun x => v (Sum.inl x)) +  $\omega$  b (fun x
  => v (Sum.inr x))
| plus a b, v => max ( $\omega$  a (fun x => v (Sum.inl x))) ( $\omega$  b
  (fun x => v (Sum.inr x)))

abbrev N (t : TCode) : Type := ACode.W t.toA

def PathSums {V : Type*} [AddCommMonoid V] : (t : TCode)  $\rightarrow$ 
  (N t  $\rightarrow$  V)  $\rightarrow$  Set V
| one, y => {0, y ACode.pt}
| plus s t, y => PathSums s (fun x => y (Sum.inl x))  $\cup$ 
  PathSums t (fun x => y (Sum.inr x))
| cone t, y => insert 0 ((fun S => y (root t) + S)) ''
  PathSums t (fun x => y (Sum.inl x)))

def IsPathInd : (t : TCode)  $\rightarrow$  (N t  $\rightarrow \mathbb{R}$ )  $\rightarrow$  Prop
| TCode.one, a => a = 0  $\vee$  a = fun _ => 1
| TCode.plus s u, a =>
  ( $\exists$  a1 : N s  $\rightarrow \mathbb{R}$ , IsPathInd s a1  $\wedge$  a = Sum.elim a1 (0
    : N u  $\rightarrow \mathbb{R}$ ))  $\vee$ 
  ( $\exists$  a2 : N u  $\rightarrow \mathbb{R}$ , IsPathInd u a2  $\wedge$  a = Sum.elim (0 :
    N s  $\rightarrow \mathbb{R}$ ) a2)
| TCode.cone u, a => a = 0  $\vee$ 
   $\exists$  a' : N u  $\rightarrow \mathbb{R}$ , IsPathInd u a'  $\wedge$  a = Sum.elim a' (fun
    _ : W ACode.one => (1 :  $\mathbb{R}$ ))

theorem clique_le_path :  $\forall$  (t : TCode) (c : N t  $\rightarrow \mathbb{R}$ ), ( $\forall$  x,
  c x = 0  $\vee$  c x = 1)  $\rightarrow$ 
  ( $\forall$  x y, c x = 1  $\rightarrow$  c y = 1  $\rightarrow$  ACode.coh t.toA x y)  $\rightarrow$   $\exists$  a,
  IsPathInd t a  $\wedge$  c  $\leq$  a

theorem mem_P_iff (t : TCode) (K : ACode) (Z : N t  $\rightarrow$  W K  $\rightarrow$ 
   $\mathbb{R}$ ) (hZ :  $\forall$  x k, 0  $\leq$  Z x k) :
  (fun p : N t  $\times$  W K => ENNReal.ofReal (Z p.1 p.2))  $\in$ 
  P (Fm.neg (Fm.tens t.toA.toFm K.toFm))  $\leftrightarrow$ 
   $\forall$  S  $\in$  PathSums t Z,  $\omega$  K S  $\leq$  1

```

```

def PMix (S : Set E) : Set E :=
  {x | ∃ (ι : Type) ( _ : Fintype ι) (w : ι → ℝ) (s : ι → E)
    (∑ i, 0 < w i) ∧ ∑ i, w i = 1 ∧ (∀ i, s i ∈ S) ∧ x = ∑
      i, w i • s i}

def IsStab (a : ACode) (v : W a → ℝ) : Prop := (∀ x, v x =
  0 ∨ v x = 1) ∧ ω a v ≤ 1

def RootSet (K : ACode) (t : TCode) : Set ((W K → ℝ) × (N t
  → W K → ℝ)) :=
  {p | IsStab K p.1 ∧ 0 ≤ p.2 ∧ ∀ S ∈ PathSums t p.2, ω K (
    p.1 + S) ≤ 1}

theorem root_step : ∀ (K : ACode) (t : TCode) (m : W K → ℝ)
  (y : N t → W K → ℝ),
  0 ≤ m → 0 ≤ y → (∀ S ∈ PathSums t y, ω K (m + S) ≤ 1) →
  (m, y) ∈ PMix (RootSet K t)

def Valid (t : TCode) (K : ACode) : Set (N t → W K → ℝ) :=
  {σ | (∀ x h, σ x h = 0 ∨ σ x h = 1) ∧ ∀ S ∈ PathSums t σ,
    IsStab K S}

theorem main : ∀ (t : TCode) (K : ACode) (z : N t → W K → ℝ)
  , 0 ≤ z →
  (∀ S ∈ PathSums t z, ω K S ≤ 1) → z ∈ PMix (Valid t K)

theorem second_order_noGap (t : TCode) (K : ACode) :
  recipe (Fm.neg (Fm.tens t.toA.toFm K.toFm)) = P (Fm.neg
    (Fm.tens t.toA.toFm K.toFm))

theorem exact_absorb : ∀ (K : ACode) {b v : W K → ℝ} {L : ℝ}
  , 0 ≤ b → 0 ≤ v → ω K b ≤ L →
  L ≤ ω K (b + v) → ∃ a : W K → ℝ, 0 ≤ a ∧ a ≤ v ∧ ω K (b
    + a) = L

def AbsOK (K : ACode) (L : ℝ) (m : W K → ℝ) (S : (W K → ℝ)
  × (W K → ℝ)) : Prop :=
  ω K (m + S.1) ≤ L ∧ ω K S.2 + L ≤ max L (ω K (m + S.1 + S
    .2))

def AbsAt (K : ACode) (t : TCode) (L : ℝ) (m : W K → ℝ) (y
  : N t → W K → ℝ) : Prop :=
  ∃ p q : N t → W K → ℝ, 0 ≤ p ∧ 0 ≤ q ∧ p + q = y ∧
  ∀ S ∈ PathSums t (fun x => (p x, q x)), AbsOK K L m S

theorem greedy (K : ACode)
  (abs0 : ∀ (t : TCode) (m : W K → ℝ) (y : N t → W K → ℝ)
    , 0 ≤ m → 0 ≤ y →

```

```

    AbsAt K t ( $\omega$  K m) m y) :
 $\forall$  (t : TCode) (L :  $\mathbb{R}$ ) (m : W K  $\rightarrow$   $\mathbb{R}$ ) (y : N t  $\rightarrow$  W K  $\rightarrow$   $\mathbb{R}$ )
    , 0  $\leq$  m  $\rightarrow$  0  $\leq$  y  $\rightarrow$   $\omega$  K m  $\leq$  L  $\rightarrow$ 
    AbsAt K t L m y

theorem absorb :  $\forall$  (K : ACode) (t : TCode) (L :  $\mathbb{R}$ ) (m : W K
 $\rightarrow$   $\mathbb{R}$ ) (y : N t  $\rightarrow$  W K  $\rightarrow$   $\mathbb{R}$ ),
0  $\leq$  m  $\rightarrow$  0  $\leq$  y  $\rightarrow$   $\omega$  K m  $\leq$  L  $\rightarrow$  AbsAt K t L m y

theorem second_order_iff (a b : ACode) :
  recipe (Fm.neg (Fm.tens a.toFm b.toFm)) = P (Fm.neg (Fm
    .tens a.toFm b.toFm))  $\leftrightarrow$ 
     $\neg$  (HasC4 a  $\wedge$  HasC4 b)

abbrev X3 : Fm := wth (base 2) (base 1)

abbrev F3 : Fm := Fm.neg (tens X3 S2)

theorem F3_eq : F3 = lolli X3 (core 2 2 2 2) := rfl

def w0 : web S2 := ((0, 0), (0, 0))
def w1 : web S2 := ((0, 0), (0, 1))
def w2 : web S2 := ((0, 1), (1, 1))
def w3 : web S2 := ((1, 0), (0, 1))
def w4 : web S2 := ((1, 1), (1, 0))

def p0 : web X3 := Sum.inl 0
def p1 : web X3 := Sum.inl 1
def pu : web X3 := Sum.inr 0

def supp3 : Finset (web F3) := {(p0, w2), (p0, w4), (p1, w0),
  (pu, w1), (pu, w3)}

noncomputable def y3 : web F3  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  := fun v => if v  $\in$ 
  supp3 then  $2^{-1}$  else 0

theorem y3_mem : y3  $\in$  P F3

theorem sharp3_le {s : web F3  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ } (hs : s  $\in$  P F3) (h01
  :  $\forall v, s v = 0 \vee s v = 1$ ) :
   $\sum v \in \text{supp3}, s v \leq 2$ 

theorem gap_F3 : recipe F3  $\subset$  P F3

theorem X3_no_C4 (Q : C4 X3) : False

```

## A.6 Other Formulas

```

noncomputable def push (j : E → F) (x : E → ℝ≥0∞) : F → ℝ≥0∞ :=
  fun a => ∑ d, if j d = a then x d else 0

structure Retract (D A : Fm) where
  j : web D → web A
  inj : Function.Injective j
  push_mem : ∀ x ∈ P D, push j x ∈ P A
  pull_mem : ∀ z ∈ P A, (fun d => z (j d)) ∈ P D

def recipe (A : Fm) : Set (web A → ℝ≥0∞) :=
  ConstructiveProb.Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s = 1}

theorem Retract.recipe_ne {D A : Fm} (R : Retract D A) (h :
  recipe D ≠ P D) :
  recipe A ≠ P A

theorem pull_push {j : E → F} (hj : Function.Injective j) (
  x : E → ℝ≥0∞) :
  (fun d => push j x (j d)) = x

theorem pairing_push (j : E → F) (u : E → ℝ≥0∞) (z : F → ℝ≥0∞) :
  pairing (push j u) z = pairing u (fun d => z (j d))

def Retract.refl (D : Fm) : Retract D D

def Retract.trans {D A C : Fm} (R : Retract D A) (S :
  Retract A C) : Retract D C

def Retract.neg {D A : Fm} (R : Retract D A) : Retract (Fm.
  neg D) (Fm.neg A)

def Retract.negneg (D : Fm) : Retract D (Fm.neg (Fm.neg D))

def Retract.tensL (D B : Fm) (b : web B) : Retract D (Fm.
  tens D B)

def Retract.tensR (D B : Fm) (b : web B) : Retract D (Fm.
  tens B D)

def Retract.wthL (D B : Fm) : Retract D (Fm.wth D B)

def Retract.wthR (D B : Fm) : Retract D (Fm.wth B D)

theorem push_tensL {X Y : Type} [Fintype X] [DecidableEq X]
  [DecidableEq Y] (b : Y)

```

```

(x : X → ℝ≥0∞) : push (fun d => (d, b)) x = tvec x (Pi.
  single b 1)

theorem push_tensR {X Y : Type} [Fintype X] [DecidableEq X]
  [DecidableEq Y] (b : Y)
  (x : X → ℝ≥0∞) : push (fun d => (b, d)) x = tvec (Pi.
    single b 1) x

theorem push_comp [Fintype F] (j1 : E → F) (j2 : F → G) (x
  : E → ℝ≥0∞) :
  push (fun d => j2 (j1 d)) x = push j2 (push j1 x)

theorem push_id [DecidableEq E] (x : E → ℝ≥0∞) : push (fun
  d : E => d) x = x

def NE : Fm → Prop
| base n => 0 < n
| Fm.neg A => NE A
| tens A B => NE A ∧ NE B
| wth A B => NE A ∧ NE B

theorem NE.point : ∀ A : Fm, NE A → Nonempty (web A)
| base _, h => ⟨⟨0, h⟩⟩
| Fm.neg A, h => NE.point A h
| tens A B, h => by
  obtain ⟨a⟩ := NE.point A h.1
  obtain ⟨b⟩ := NE.point B h.2
  exact ⟨(a, b)⟩
| wth A _, h => by
  obtain ⟨a⟩ := NE.point A h.1
  exact ⟨Sum.inl a⟩

/-- 'Occ D p A': 'D' occurs in 'A', positively if 'p' and
  negatively otherwise. -/
inductive Occ (D : Fm) : Bool → Fm → Prop
| self : Occ D true D
| neg {p : Bool} {A : Fm} : Occ D p A → Occ D (!p) (Fm.
  neg A)
| tensL {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D
  p (tens A B)
| tensR {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D
  p (tens B A)
| wthL {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D p
  (wth A B)
| wthR {p : Bool} {A : Fm} (B : Fm) : Occ D p A → Occ D p
  (wth B A)

/-- 'D' or 'D⊥', according to polarity. -/
def pol : Bool → Fm → Fm

```

```

| true, D => D
| false, D => Fm.neg D

/-- **Occurrences give retracts.** -/
theorem occ_retract {D : Fm} :  $\forall$  {p : Bool} {A : Fm}, Occ D
  p A  $\rightarrow$  NE A  $\rightarrow$ 
    Nonempty (Retract (pol p D) A)
| _, _, Occ.self, _ => <Retract.refl D>
| _, _, Occ.neg (p := p) h, hA => by
  obtain <R> := occ_retract h hA
  cases p
  · exact <(Retract.negneg D).trans R.neg>
  · exact <R.neg>
| _, _, Occ.tensL B h, hA => by
  obtain <R> := occ_retract h hA.1
  obtain <b> := NE.point B hA.2
  exact <R.trans (Retract.tensL _ B b)>
| _, _, Occ.tensR B h, hA => by
  obtain <R> := occ_retract h hA.2
  obtain <b> := NE.point B hA.1
  exact <R.trans (Retract.tensR _ B b)>
| _, _, Occ.wthL B h, hA => by
  obtain <R> := occ_retract h hA.1
  exact <R.trans (Retract.wthL _ B)>
| _, _, Occ.wthR B h, hA => by
  obtain <R> := occ_retract h hA.2
  exact <R.trans (Retract.wthR _ B)>

/-- The second-order core ' $(n \multimap m) \otimes (n' \multimap m')$ '. -/
abbrev core (n m n' m' :  $\mathbb{N}$ ) : Fm := tens (lolli (base n) (
  base m)) (lolli (base n') (base m'))

/-- **The gap at every formula with a negative occurrence
of the core.** -/
theorem gap_of_occ {n m n' m' :  $\mathbb{N}$ } (hn :  $2 \leq n$ ) (hm :  $2 \leq m$ )
  (hn' :  $2 \leq n'$ ) (hm' :  $2 \leq m'$ )
  {A : Fm} (hocc : Occ (core n m n' m') false A) (hA : NE
    A) : recipe A  $\subset$  P A

example : recipe (lolli (core 2 2 2 2) (base 2))  $\subset$  P (lolli
  (core 2 2 2 2) (base 2))

theorem push_tvec {X Y X' Y' : Type} [Fintype X] [Fintype Y]
  [DecidableEq X'] [DecidableEq Y']
  (j : X  $\rightarrow$  X') (k : Y  $\rightarrow$  Y') (x : X  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) (y : Y  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ )
  :
  push (fun d : X  $\times$  Y => (j d.1, k d.2)) (tvec x y) =
    tvec (push j x) (push k y)

```

```

def Retract.tens {D A D' A' : Fm} (R : Retract D A) (R' :
  Retract D' A') :
  Retract (Fm.tens D D') (Fm.tens A A')

theorem gap_of_retracts {n m n' m' : ℕ} (hn : 2 ≤ n) (hm :
  2 ≤ m) (hn' : 2 ≤ n') (hm' : 2 ≤ m')
  {X Y : Fm} (RX : Retract (lolli (base n) (base m)) X)
  (RY : Retract (lolli (base n') (base m')) Y) :
  recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg (tens X Y))

theorem coh_symm : ∀ (A : Fm) {s t : web A}, coh A s t →
  coh A t s

structure C4 (X : Fm) where
  (a b c d : web X)
  hab : coh X a b
  hbc : coh X b c
  hcd : coh X c d
  hda : coh X d a
  hac : ¬ coh X a c
  hbd : ¬ coh X b d

def C4.j (Q : C4 X) : Fin 2 × Fin 2 → web X := fun p =>
  ![![Q.a, Q.c], ![Q.b, Q.d]] p.1 p.2

def C4.retract (Q : C4 X) : Retract (lolli (base 2) (base
  2)) X

theorem gap_of_C4 {X Y : Fm} (QX : C4 X) (QY : C4 Y) :
  recipe (Fm.neg (tens X Y)) ⊂ P (Fm.neg (tens X Y))

theorem gap_of_C4_occ {X Y A : Fm} (QX : C4 X) (QY : C4 Y)
  (hocc : Occ (tens X Y) false A)
  (hA : NE A) : recipe A ⊂ P A

def C4.lolli22 : C4 (lolli (base 2) (base 2))

def C4.curried : C4 (Fm.neg (lolli (lolli (base 2) (base 2)
  ) (base 2)))

theorem gap_curried :
  recipe (lolli (lolli (base 2) (base 2)) (lolli (lolli (
    base 2) (base 2)) (base 2)))
    ⊂ P (lolli (lolli (base 2) (base 2)) (lolli (lolli (
      base 2) (base 2)) (base 2)))

```

## A.7 Local Certainty and Duality

```

def locS (C : Set (Finset E)) : Set (E → ℝ≥0∞) := {x | ∀ t
  ∈ C, ∑ e ∈ t, x e ≤ 1}

def locN (C : Set (Finset E)) : Set (E → ℝ≥0∞) := {x | ∀ t
  ∈ C, ∑ e ∈ t, x e = 1}

def globS (C : Set (Finset E)) : Set (E → ℝ≥0∞) := Mix {x |
  x ∈ locS C ∧ IsSharpVec x}

def globN (C : Set (Finset E)) : Set (E → ℝ≥0∞) := Mix {x |
  x ∈ locN C ∧ IsSharpVec x}

theorem globN_sub (C : Set (Finset E)) : globN C ⊆ locN C

theorem globS_sub (C : Set (Finset E)) : globS C ⊆ locS C

theorem globN_single : globN ({Finset.univ} : Set (Finset E
)) = locN {Finset.univ}

theorem globS_single : globS ({Finset.univ} : Set (Finset E
)) = locS {Finset.univ}

def loc (A : Fm) : Set (web A → ℝ≥0∞) := orth (sharpP (Fm.
neg A))

def dualCtx (A : Fm) : Set (Finset (web A)) :=
  {t | ∃ s ∈ sharpP (Fm.neg A), t = Finset.univ.filter fun
    e => s e = 1}

theorem loc_eq_locS (A : Fm) : loc A = locS (dualCtx A)

theorem recipe_sub_P_sub_loc (A : Fm) : recipe A ⊆ P A ∧ P
  A ⊆ loc A

theorem P_eq_loc_iff (A : Fm) : P A = loc A ↔ recipe (Fm.
neg A) = P (Fm.neg A)

theorem clique_mem_loc (A : Fm) {c : web A → ℝ≥0∞} (h01 : ∀
  e, c e = 0 ∨ c e = 1)
  (hc : ∀ x y, c x = 1 → c y = 1 → coh A x y) : c ∈ loc A

theorem P_S2_eq_loc : P S2 = loc S2

theorem P_TT_subset_loc : P TT ⊆ loc TT

def φ : W × W → web S2 := emb (boolFin le_rfl) (boolFin
  le_rfl) (boolFin le_rfl) (boolFin le_rfl)

abbrev S2 : Fm := second 2 2 2 2

```

```

abbrev TT : Fm := neg (tens S2 S2)

noncomputable def shannonF : Finset (web (tens S2 S2)) :=
  shannon.image fun u => ( $\varphi$  u.1,  $\varphi$  u.2)

theorem shannonF_clique :  $\forall u \in \text{shannonF}, \forall v \in \text{shannonF},$ 
  coh TT u v

noncomputable def pentS2 : web S2  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
  pentE (boolFin le_rfl) (boolFin le_rfl) (boolFin le_rfl)
  (boolFin le_rfl)

theorem shannonF_not_mem : (fun u => if u  $\in$  shannonF then
  (1 :  $\mathbb{R}_{\geq 0\infty}$ ) else 0)  $\notin$  P TT

abbrev W2 := (W  $\times$  W)  $\times$  (W  $\times$  W)

noncomputable def tens2 (t t' : W  $\times$  W  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : W2  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
  := fun u => t u.1 * t' u.2

def gensTT : Set (W2  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) := {u |  $\exists t \in \text{tensDual}, \exists t' \in$ 
  tensDual, u = tens2 t t'}

def PTT : Set (W2  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) := orth (orth gensTT)
def PTTdual : Set (W2  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) := orth gensTT

def cohTT : W2  $\rightarrow$  W2  $\rightarrow$  Prop := tensC secondC secondC
def cohTTdual : W2  $\rightarrow$  W2  $\rightarrow$  Prop := dualC cohTT

def shannon : Finset W2 :=
  {(((true, true), (true, true)), ((true, true), (true,
    true))),
    (((false, true), (false, true)), ((false, true), (true,
    false))),
    (((false, true), (true, false)), ((false, false), (false,
    false))),
    (((true, false), (false, false)), ((false, true), (false,
    true))),
    (((false, false), (false, false)), ((true, false), (
    false, false)))}

theorem shannon_clique : IsClique cohTTdual ( $\cdot \in \text{shannon}$ )

theorem shannon_not_mem :
  (fun u => if u  $\in$  shannon then (1 :  $\mathbb{R}_{\geq 0\infty}$ ) else 0)  $\notin$ 
  PTTdual

```

```

theorem orth_recipe (A : Fm) : orth (recipe A) = loc (Fm.
  neg A)

theorem orth_loc (A : Fm) : orth (loc A) = recipe (Fm.neg A
)

theorem tight_iff (A : Fm) :
  (orth (recipe A) = recipe (Fm.neg A) ∧ orth (recipe (Fm.
    .neg A)) = recipe A) ↔
  (recipe A = P A ∧ recipe (Fm.neg A) = P (Fm.neg A))

theorem tight_iff_recipe_eq_loc (A : Fm) :
  orth (recipe A) = recipe (Fm.neg A) ↔ recipe A = loc A

theorem not_tight_core : orth (recipe (core 2 2 2 2)) ≠
  recipe (Fm.neg (core 2 2 2 2))

theorem P_lolli_eq_loc (n m : ℕ) :
  P (lolli (base n) (base m)) = loc (lolli (base n) (base
    m))

theorem tight_lolli (n m : ℕ) :
  orth (recipe (lolli (base n) (base m))) = recipe (Fm.
    neg (lolli (base n) (base m)))

theorem tight_not_tensor_closed {n m n' m' : ℕ} (hn : 2 ≤ n
) (hm : 2 ≤ m) (hn' : 2 ≤ n')
(hm' : 2 ≤ m') :
  orth (recipe (lolli (base n) (base m))) = recipe (Fm.
    neg (lolli (base n) (base m))) ∧
  orth (recipe (lolli (base n') (base m'))) = recipe (Fm.
    neg (lolli (base n') (base m'))) ∧
  orth (recipe (tens (lolli (base n) (base m)) (lolli (
    base n') (base m'))))
  ≠ recipe (Fm.neg (tens (lolli (base n) (base m)) (
    lolli (base n') (base m'))))

theorem sharp_dual_exclusive {s : web (core 2 2 2 2) → ℝ≥0∞
}
  (hs : s ∈ sharpP (Fm.neg (core 2 2 2 2))) {e e' : BEv}
  (hne : e ≠ e') (he : s e = 1)
  (he' : s e' = 1) : ¬ cohE e e'

theorem chsh_bilinear {X Y : Fin 2 × Fin 2 → ℝ≥0∞} (hX : ∀
  a, ∑ b, X (a, b) ≤ 1)
  (hY : ∀ a, ∑ b, Y (a, b) ≤ 1) :
  ∑ e : BEv, (if win e then X e.1 * Y e.2 else 0) ≤ 3

theorem prBox_mem_loc : prBox ∈ loc (core 2 2 2 2)

```

```

theorem prBox_not_mem_P : prBox  $\notin$  P (core 2 2 2 2)

theorem P_core_ssubset_loc : P (core 2 2 2 2)  $\subset$  loc (core 2
  2 2 2)

```

## A.8 Quantum Event Logic

```

def vec : Fin 18  $\rightarrow$  Fin 4  $\rightarrow$   $\mathbb{Z}$  := ![
  ![-1, 1, 1, 1], ![0, 0, 0, 1], ![0, 0, 1, 0], ![0, 0, 1,
    1], ![0, 1, -1, 0], ![0, 1, 0, -1],
  ![0, 1, 0, 0], ![1, -1, -1, 1], ![1, -1, 0, 0], ![1, -1,
    1, -1], ![1, 0, -1, 0], ![1, 0, 0, -1],
  ![1, 0, 0, 1], ![1, 0, 1, 0], ![1, 1, -1, 1], ![1, 1, 0,
    0], ![1, 1, 1, -1], ![1, 1, 1, 1]]

def bas : Fin 9  $\rightarrow$  Fin 4  $\rightarrow$  Fin 18 := ![
  ![1, 2, 15, 8], ![1, 6, 13, 10], ![9, 7, 15, 3], ![9, 17,
    10, 5], ![2, 6, 12, 11],
  ![7, 17, 11, 4], ![14, 16, 8, 3], ![14, 0, 13, 5], ![16,
    0, 12, 4]]

def dot (u v : Fin 4  $\rightarrow$   $\mathbb{Z}$ ) :  $\mathbb{Z}$  :=  $\sum$  k, u k * v k

theorem basis_orth :  $\forall$  b : Fin 9,  $\forall$  i j : Fin 4, i  $\neq$  j  $\rightarrow$ 
  dot (vec (bas b i)) (vec (bas b j)) = 0

theorem occ_two :  $\forall$  v : Fin 18,
  (univ.filter (fun p : Fin 9  $\times$  Fin 4 => bas p.1 p.2 = v)
    ).card = 2

theorem no_ks :  $\neg \exists$  f : Fin 18  $\rightarrow$  Bool,  $\forall$  b, (univ.filter (
  fun i => f (bas b i))).card = 1

def certain : Set (Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) :=
  {s | IsSharpVec s  $\wedge \forall$  b,  $\sum$  i, s (bas b i) = 1}

theorem no_certain_state : certain =  $\emptyset$ 

theorem Mix_certain_empty : Mix certain =  $\emptyset$ 

theorem parseval ( $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ ) (b : Fin 9) :
   $\sum$  i, ( $\sum$  k,  $\psi$  k * vec (bas b i) k)  $^2$  /  $\sum$  k, ((vec (bas
    b i) k :  $\mathbb{R}$ ))  $^2$  =  $\sum$  k,  $\psi$  k  $^2$ 

def states : Set (Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) := {x |  $\forall$  b,  $\sum$  i, x (bas b
  i) = 1}

```

```

noncomputable def born ( $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ ) : Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  :=
  fun v =>
    ENNReal.ofReal (( $\sum k$ ,  $\psi k * \text{vec } v k$ )  $\wedge$  2 /  $\sum k$ , ((vec v k
      :  $\mathbb{R}$ ))  $\wedge$  2)

theorem born_mem { $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ } ( $h\psi$  :  $\sum k$ ,  $\psi k \wedge 2 = 1$ ) :
  born  $\psi \in \text{states}$ 

theorem born_not_mix { $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ } ( $h\psi$  :  $\sum k$ ,  $\psi k \wedge 2 =$ 
  1) :
  born  $\psi \in \text{states} \wedge \text{born } \psi \notin \text{Mix certain}$ 

def partialCertain : Set (Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) :=
  {s | IsSharpVec s  $\wedge \forall b$ ,  $\sum i$ , s (bas b i)  $\leq 1$ }

theorem sum_bases (f : Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) :
   $\sum b$  : Fin 9,  $\sum i$  : Fin 4, f (bas b i) = 2 *  $\sum v$ , f v

theorem partial_sum_le_four {s : Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ } (hs : s  $\in$ 
  partialCertain) :  $\sum v$ , s v  $\leq 4$ 

theorem mix_sum_le_four {x : Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ } (hx : x  $\in$  Mix
  partialCertain) :  $\sum v$ , x v  $\leq 4$ 

theorem born_total { $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ } ( $h\psi$  :  $\sum k$ ,  $\psi k \wedge 2 = 1$ )
  : 2 *  $\sum v$ , born  $\psi v = 9$ 

theorem born_not_mix_partial { $\psi$  : Fin 4  $\rightarrow$   $\mathbb{R}$ } ( $h\psi$  :  $\sum k$ ,  $\psi k$ 
   $\wedge 2 = 1$ ) :
  born  $\psi \notin \text{Mix partialCertain}$ 

def S4 : Finset (Fin 18) := {0, 1, 3, 11}

theorem S4_stable :  $\forall b$  : Fin 9, (Finset.univ.filter (fun i
  : Fin 4 => bas b i  $\in$  S4)).card  $\leq 1$ 

noncomputable def ind4 : Fin 18  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  := fun v => if v  $\in$ 
  S4 then 1 else 0

theorem ind4_mem : ind4  $\in$  partialCertain  $\wedge \sum v$ , ind4 v = 4

def ctx : Set (Finset (Fin 18)) := {t |  $\exists b$ , t = Finset.
  univ.image (bas b)}

theorem locN_cabello : locN ctx = states

theorem globN_cabello : globN ctx =  $\emptyset$ 

```

```

theorem born_local {ψ : Fin 4 → ℝ} (hψ : ∑ k, ψ k ^ 2 = 1)
  : born ψ ∈ locN ctx

theorem born_mono (ψ : Fin 4 → ℝ) : born ψ 1 ≤ born ψ 12 +
  born ψ 11

def halfSet : Finset (Fin 18) := {0, 1, 2, 6, 7, 9, 14, 16,
  17}

noncomputable def half : Fin 18 → ℝ≥0∞ := fun v => if v ∈
  halfSet then 2-1 else 0

theorem half_not_born :
  half ∈ locN ctx ∧ half ∉ Mix {x | ∃ ψ : Fin 4 → ℝ, ∑ k,
    ψ k ^ 2 = 1 ∧ x = born ψ}

noncomputable def unit (u : Fin 18) : Fin 4 → ℝ :=
  fun k => (vec u k : ℝ) / Real.sqrt (dot (vec u) (vec u))

theorem unit_norm (u : Fin 18) : ∑ k, unit u k ^ 2 = 1

theorem born_ray_zero_iff (u v : Fin 18) : born (unit u) v
  = 0 ↔ dot (vec u) (vec v) = 0

theorem born_ray_one_iff (u v : Fin 18) : born (unit u) v =
  1 ↔ u = v

```